

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

PATRON ARQUITECTONICO: Cliente-Servidor con API RESTful

Se implementa un patrón MVC ligero usando Flask como backend y HTML/JS vanilla como frontend desacoplado. La separación de responsabilidades es clara: lógica de negocio en Python, presentación en HTML/CSS, interacción asíncrona en JavaScript.

CAPAS DEL SISTEMA:

1. Capa de Presentación: index.html con formulario reactivo, validación client-side y fetch API para comunicación asíncrona.
2. Capa de API: Flask RESTful exponiendo endpoint POST /api/validate-rut con respuestas JSON estandarizadas.
3. Capa de Lógica de Negocio: Módulo Python puro con algoritmo de validación del dígito verificador chileno (módulo 11).
4. Capa de Auditoría: Sistema de logging estructurado para registrar intentos de validación inválidos.

SEGURIDAD (DevSecOps):

- Sanitización de entrada: regex estricto que permite solo dígitos y guion antes de procesar.
- Protección contra XSS: escape de salidas en templates Jinja2 y Content-Security-Policy headers.
- Rate Limiting: uso de Flask-Limiter para prevenir abuso del endpoint de validación.
- CORS controlado: solo orígenes permitidos pueden consumir la API.
- Logging de seguridad: registro de IPs, timestamps y payloads inválidos en archivo de auditoría rotativo.
- Variables de entorno: configuración sensible via .env con python-dotenv.
- Headers de seguridad HTTP: X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security.

TECNOLOGIAS:

- Backend: Python 3.11+, Flask 3.x, Flask-Limiter, python-dotenv
- Frontend: HTML5, CSS3 (custom + responsive), JavaScript ES6+ (Fetch API)
- Logging: módulo logging nativo de Python con RotatingFileHandler
- Testing: pytest + pytest-flask para unit e integration tests
- Documentación API: comentarios OpenAPI/Swagger inline

ALGORITMO RUT CHILENO:

Implementación del módulo 11 estándar: se multiplica cada dígito del RUT (de derecha a izquierda) por la secuencia 2,3,4,5,6,7,2,3... se suman los productos, se divide por 11, el resto determina el dígito verificador (0=0, 1=K, resto=11-resto).

FLUJO DE VALIDACION:

1. Cliente envía POST /api/validate-rut con body JSON {rut: string}
2. Servidor sanitiza y valida formato con regex
3. Si formato inválido: log de auditoría + respuesta 400 con mensaje claro
4. Si formato válido: ejecuta algoritmo módulo 11
5. Retorna JSON {valid: bool, message: string, rut_formateado: string}
6. Cliente muestra resultado sin recargar página

2. MÉTRICAS DE ITERACIÓN (QA)

- run.py | Estado: APROBADO | Iteraciones: 1
- app/__init__.py | Estado: APROBADO | Iteraciones: 1
- app/routes/__init__.py | Estado: APROBADO | Iteraciones: 1
- app/routes/rut_routes.py | Estado: APROBADO | Iteraciones: 1
- app/services/__init__.py | Estado: APROBADO | Iteraciones: 1
- app/services/rut_validator.py | Estado: APROBADO | Iteraciones: 2
- app/utils/__init__.py | Estado: APROBADO | Iteraciones: 1

- app/utills/sanitizer.py | Estado: APROBADO | Iteraciones: 1
- app/utills/logger.py | Estado: APROBADO | Iteraciones: 1
- app/middleware/__init__.py | Estado: APROBADO | Iteraciones: 1
- app/middleware/security_headers.py | Estado: APROBADO | Iteraciones: 1
- config/__init__.py | Estado: APROBADO | Iteraciones: 1
- config/settings.py | Estado: WARNING | Iteraciones: 3
- templates/index.html | Estado: APROBADO | Iteraciones: 1
- static/css/styles.css | Estado: APROBADO | Iteraciones: 1
- static/js/rut_handler.js | Estado: APROBADO | Iteraciones: 1
- tests/__init__.py | Estado: APROBADO | Iteraciones: 1
- tests/test_rut_validator.py | Estado: APROBADO | Iteraciones: 1
- tests/test_routes.py | Estado: APROBADO | Iteraciones: 1
- tests/test_sanitizer.py | Estado: APROBADO | Iteraciones: 1
- logs/.gitkeep | Estado: APROBADO | Iteraciones: 1
- docs/api_documentation.md | Estado: APROBADO | Iteraciones: 1
- .env.example | Estado: APROBADO | Iteraciones: 1
- .gitignore | Estado: APROBADO | Iteraciones: 1
- requirements.txt | Estado: APROBADO | Iteraciones: 1
- requirements-dev.txt | Estado: APROBADO | Iteraciones: 2
- README.md | Estado: APROBADO | Iteraciones: 1